

Assignment 02

C++ Object-oriented Programming

Khiem Nguyen Lecturer in Multiscale Materials khiem.nguyen@glasgow.ac.uk

Table of Content

1 Foreword	1
1.1 Structure of the paper exam	1
1.2 How to pass the course	1
1.3 What to review	2
2 Mock exam	3
2.1 Question 1: C++ Theory	3
Question (a)	3
Answer (a)	4
Question (b)	4
Answer (b)	6
Question (c)	6
Answer (c)	7
Question (d)	8
Answer (d)	9
2.2 Question 2: Writing C++	9
Problem description	9
Answer in Code	10
2.3 Question 3: UML Class Diagram	12
Problem description	12
Answer	13

1 Foreword

1.1 Structure of the paper exam

The real paper exam contains 3 questions with the following grade distribution:

Question	Grade
Q1: C++ Theory	30
Q2: Writing C++ Code	40
Q3: UML Class diagram	30

1.2 How to pass the course

Over the the past two years I received two common questions. I believe that it is good to write down my answers so that you understand the course's demand, and my philosophy.

- **Question 1:** Can I pass the entire course with 0 on the paper exam because I have already achieved 80% on the 50% of the course grade (coursework as continuous assessment)

Answer: No, you won't. According to the UoG's guideline, you need to pass any component constituting of more than 30% of the course grade by the minimum **E3** (namely 30 points out of 100 points). In general, I am happy to pass somebody with 28–29 points with true effort. But if that somebody got 28 points due to my gentle marking, I still need to fail them. For example, I give 1 point for every attempt while the answers are all wrong and your grade still cannot cross the threshold 30 points, you deserve a fail even at the boundary 29 points.

- **Question 2:** Can I pass the exam by learning everything by heart without understanding the logics or should I understand the logic?

Answer: This question sounds irony but it is completely legitimate. Here is my thought. First, exams are designed to test your knowledge, not to test your memory. So, without understanding the logics, I think you will likely fail. Second, I don't believe that anybody can learn 200 lines of code by heart and can apply what they memorize to another applications without understanding a single thing. In doing that, you have already understood something. If you believe that you already learned 200 lines of code by heart but you could not apply what you did to a similar application, then you actually did not learn anything by heart.

1.3 What to review

When I was a student, my professors always answer the question “what to review?” by a single sentence: “Everything we have learned so far”. That's all, no further elaboration. Personally, I think this should be a standard question for such a question too. What's the point if you can pass an exam by knowing a very specific area. You are educated to work in real-world environment, not to pass the exams.

Saying that, this is my advice. You should at least understand the following important concepts and know how to implement them in C++ code.

- **Basics of C++:** To understand the code and to understand exam's questions, of course you need to know and understand the basics of C++.
 - **Iteration statements:** for loop, while loop, do-while loop
 - **Condition statements:** if statement, if-else statement, if-else-if statement
 - **Functions:** function definition, parameters versus arguments, return value of a function
- **Intermediate of C++:** The question of C++ theory basically may ask you anything we have learned so far from the lectures. However, these are the concepts you may want to pay good attention on
 - **reference**, *pass by reference* versus *pass by value* in writing functions
 - **pointer**, *pass by pointer* (frequently called *pass by address*)
 - **template:** function template, class template
- **Object-oriented programming:** Of course, you should be able to implement very basics of object-oriented programming paradigm.
 - **class:** define a class, write a constructor, write a destructor, list initializer for constructor, etc.
 - **access specifiers:** private, public and protected. protected is rarely used but you may at least need to be aware of this.
 - **overriding operators:** override operator << (output the object), arithmetic operators + (addition), - (subtraction), * (multiplication), etc.
 - **inheritance:** basic concept of inheritance, how to define a child class
 - **polymorphism:** basic concept of polymorphism, how to implement polymorphism for children classes using virtual function.
 - **abstract class:** basic concept of abstract class, how to make a class abstract.

That sounds a lot!?!? **Not really.** If you practice **Question 2:: Writing C++ Code**, you have reviewed most of the important points here.

2 Mock exam

The **mock exam** here follows similar structure of the **real exam** with of course different questions in the part asking C++ theories and a different application in the question asking writing C++ code, and in drawing UML class diagram.

Appearance of the real exam vs the mock exam

The real paper exam is prepared by LaTeX. So you should see that the questions look like in the past paper exam, namely (a), (b), ..., (g). However, this mock exam is prepared by using Quarto. Therefore, the appearance of this mock exam may not look exactly the same as the real one although they are essentially the same. The content is tremendously helpful for the real paper exam though.

Learn the logics behind the mock exam

- Note that the **real exam** is **definitely much shorter than** the **mock exam** here. The mock exam is longer because I want to provide an opportunity for you to “really learn”, not to “remember by heart”.
- I struggle preparing this mock exam because I don’t want it to be so similar to the real paper exam. But I don’t want it to be completely different from the real exam. What’s the point of a mock exam if it is completely different?
- **So you should learn the programming logics behind this mock exam, don’t try to learn it by heart.**

2.1 Question 1: C++ Theory

Question (a)

You are given the following code snippet:

```
1  #include <iostream>
2
3  // 1. Increments the value of 'n' by 10 using a pointer.
4  void increment_by_pointer(int* ptr) {
5      // Write code here
6  }
7
8  // 2. Doubles the value of 'n' using a reference.
9  void double_by_reference(int& ref) {
10     // Write code here
11 }
12
13 // 3. Swaps the address held by two pointers (Pointer to Pointer).
14 // Note: This should change where the pointers in main point to.
15 void swap_pointers(int** ptr1, int** ptr2) {
16     // Write code here
17 }
18
19 int main() {
20     int a { 5 };
21     int b { 50 };
22
23     int* pA { &a };
24     int* pB { &b };
```

```

25
26 // Task 1: Use increment_by_pointer to make 'a' equal to 15.
27 // Task 2: Use double_by_reference to make 'b' equal to 100.
28 // Task 3: Use swap_pointers so that pA points to 'b' and pB points to 'a'.
29
30 return 0;
31 }

```

Question:

1. Write the implementation for `increment_by_pointer()`.
2. Write the implementation for `double_by_reference()`.
3. Write the implementation for `swap_pointers()`.
4. Write the three lines of code inside `main()` required to call these functions correctly using the variables provided.

You only need to missing code – don't rewrite what have been provided. You may need to write the line number shown on the code for easy navigation.

Answer (a)

You only need to write the code that the question asks to write. Don't waste time rewriting code that the question does not ask to write.. If you want to be sure, just write the line number

1. Implementation for `increment_by_pointer()`, at line 5

```
*ptr += 10;
```

Explanation: We must dereference the pointer.

2. Implementation for `double_by_reference()`, at line 10:

```
ref *= 2;
```

Explanation: We must treat the reference like standard variable

3. Implementation for `swap_pointers()`, inside the function body, from line 16:

```

int* temp = *ptr1;
*ptr1 = *ptr2;
*ptr2 = temp;

```

4. Code in the `main()` function body:

```

increment_by_pointer(&a); // or increment_by_pointer(pA);
double_by_reference(b);
swap_pointers(&pA, &pB);

```

Question (b)

You are given the following code snippet:

```

1  #include <iostream>
2  #include <string>
3
4  class Student {
5  private:
6      std::string name;
7      mutable int access_count { 0 };
8
9  public:
10     Student(std::string name) : name {name} {}
11
12     // A: Returns the name
13     std::string get_name() const {
14         access_count++;
15         return name;
16     }
17
18     // B: Sets a new name
19     void set_name(std::string name) {
20         this->name = name;
21     }
22 };
23
24 int main() {
25     Student std_01 {"Tony Stark"};
26     const Student std_02 {"Hulk"};
27
28     // Line 1: Pointer to a constant Student
29     const Student* ptr = &std_01;
30     ptr->set_name("New Name");
31
32     // Line 2: Constant pointer to a Student
33     Student* const cptr = &std_02;
34     cptr->set_name("New Name");
35
36     // Line 3: Method call
37     std::cout << std_02.get_name() << " accessed " << std::endl;
38
39     return 0;
40 }

```

Question

1. **Analyze Line 1 and Line 2:** Both lines will cause a compilation error. Explain the difference between `const Student*` and `Student* const` and why each specific line fails.
2. **Analyze Method A (`get_name`):** Even though this method is marked `const`, it modifies `access_count`. Explain why this code **will** compile and what the `mutable` keyword achieves in this context. What happens if we remove `mutable`?
3. **Analyze Line 3:** If the `const` qualifier were removed from the declaration of `get_name()`. Line 3 would fail to compile. Explain why a `const` object like `std_02` can only call `const` member functions.

Answer (b)

1. Task 1 (pointers): In C++, we should read pointer declarations from **right to left**. This tells us exactly what the **const** is “locking”.
 - Consider `const Student* ptr`:
 - Read right-to-left: `ptr` is a pointer to a `Student` that is `const`.
 - Meaning: We can move the pointer to a different student, but we cannot change the data of the student it's currently pointing to. The “Data” is frozen.

Coming back to `const Student* ptr`: This is a **pointer to a constant**. The pointer can point to another student (also a `const`), but it cannot change the data it points to. `set_name` is a non-`const` function, so it's forbidden.

- Consider `Student* const cptr`
 - Read right-to-left: `cptr` is a *const pointer* to a `Student`.
 - Meaning: The pointer itself is stuck. We can change the data of the student it points to, but we can never point it to someone else (another student). The “Address” is frozen.

Coming back to `*Student* const cptr`: This is a **constant pointer**. We cannot change where `cptr` points to (after initialization), but the `Student` object `cptr` points to can (ordinarily) change its data. However, since `std_02` is declared `const`, a non-`const` pointer cannot point to it.

2. Task 2 (mutable): Coming back to `*Student* const cptr` Since we have not studied `mutable`, this mock question is for your knowledge – no real question on this keyword.
 - `mutable` allows a member variable to be modified even within a `const` member function. This is used for “logical const-ness” (e.g. caching, debugging, or access counters) where the visible state of the object does not change.
3. Task 3 (**const objects**):
 - A `const` object represents a contract that the object's state will not change. Therefore, the compiler only permits calls to member functions that have explicitly promised not to modify the object via the `const` suffix.

Question (c)

You are given the following code snippet:

```
1  #include <iostream>
2  #include <string>
3
4  // --- Write your Function Template here ---
5
6
7  // -----
8
9  struct Box {
10     int volume;
11     // Overloading the > operator for Box comparison
12     bool operator>(const Box& other) const {
13         return volume > other.volume;
14     }
15 };
16
```

```

17 int main() {
18     // Test Case 1: Integers
19     int a { 10 }, b { 20 };
20     std::cout << "Max int: " << find_max(a, b) << '\n';
21
22     // Test Case 2: Strings
23     std::string str_01 { "Apple" }, str_02 { "Zebra" };
24     std::cout << "Max string: " << find_max(str_01, str_02) << '\n';
25
26     // Test Case 3: Custom Objects
27     Box box_01 { 500 }, box_02 { 1000 };
28     Box biggest = find_max(box_01, box_02);
29     std::cout << "Max Box volume: " << biggest.volume << '\n';
30
31     return 0;
32 }

```

Question

1. Write the full definition of the function template `find_max`. It should take two parameters of the same template type by reference to const and return the result by value.
2. In **Test Case 3**, the code compiles because the `Box` structure has overloaded `operator>`. If this operator was **not** overloaded, what specific error would the compiler generate when trying to instantiate the template?
3. If the function was called as `find_max(10, 3.14)`, the compiler would fail to deduce the template argument. Explain why this happens and provide two ways a developer could fix the call in `main()` without changing the template definition.

Answer (c)

1. Template implementation:

```

template <typename T>
T find_max(const T& x, const T& y) {
    return (x > y) ? x : y;
}

```

2. Missing operator:

- The compiler would throw an error stating that `operator>` is not defined for the type `Box`.
- Key concept: The student should mention that template requirements (concepts) are checked **at the time of instantiation**, not definition.

3. Ambiguous Deduction:

- Why: 10 is an `int` and 3.14 is a `double`. The compiler cannot decide if `T` should be `int` or `double` because the template expects both parameters cannot be the exact same type `T`.

- Fix Option 1 (explicit instantiation):

```
find_max<double>(10, 3.14);
```

- Fix Option 2 (Casting):

```
find_max(static_cast<double>(10), 3.14);
```

Question (d)

You are given the following code snippet

```
1  #include <iostream>
2  #include <string>
3
4  class Payment {
5  public:
6      void identify() {
7          std::cout << "Standard Payment" << std::endl;
8      }
9      virtual void process() {
10         std::cout << "Processing generic payment..." << std::endl;
11     }
12     virtual ~Payment() = default;
13 };
14
15 class CreditCard : public Payment {
16 public:
17     void identify() {
18         std::cout << "Credit Card" << std::endl;
19     }
20     void process() override {
21         std::cout << "Processing credit card via secure gateway..." << std::endl;
22     }
23 };
24
25 int main() {
26     Payment* payment_ptr = new CreditCard();
27     CreditCard* credit_ptr = new CreditCard();
28
29     std::cout << "1: "; payment_ptr->identify();
30     std::cout << "2: "; payment_ptr->process();
31
32     std::cout << "3: "; credit_ptr->identify();
33
34     delete payment_ptr;
35     delete credit_ptr;
36
37     return 0;
38 }
```

Question

1. **Predict the output:** Write down the exact console output for the liens marked 1, 2, and 3.
2. **Explain the difference:** Why does `payment_ptr->identify()` produce a different result style compared to `payment_ptr->process()`? Explain the role of virtual table in your answer.
3. The **override** keyword: In the `CreditCard` class, the `process` function uses the `override` keyword. Explain one specific benefit of using this keyword for a developer.
4. The **Destructor**: In the `Payment` class, the destructor is marked `virtual`. What would happen if we removed the `virtual` keyword and executed `delete payment_ptr`?

Answer (d)

1. The output on the console `{.cpp}` `Standard Payment` `Processing credit card via secure gateway...` `Credit Card`

Explanation:

- `Standard Payment` – static binding uses pointer type
- `Processing credit card via secure gateway...` – Dynamic binding uses the actual object type
- `Credit Card` – static binding on a derived pointer

2. Virtual Table explanation

- `identify()` is a non-virtual function. The compiler uses **static binding**, meaning it looks at the type of the point (`Payment*`) at **compile-time**.
- `process()` is a virtual function. The compiler uses **dynamic binding**. At **runtime**, the program follows the object's internal **vpointer** (**v** stands for virtual) to the **vtable**, which points to the `CreditCard` implementation.

3. Override benefit

- It provides a compiler check. If the signature in the derived class does not exactly match a virtual function in the base class (e.g., a typo in the name or a missing `const`), the compiler will trigger an error instead of silently creating a new function.

4. Virtual destructor: We have not learned this in the lectures, so don't worry! I explain here for your knowledge.

- If the destructor is not virtual, deleting a `Derived` object through a `Base*` results in **undefined behavior**. Usually, only the `Base` part of the object is destroyed, leading to a memory leak if the `Derived` class allocated its own resources.

2.2 Question 2: Writing C++

Problem description

In mathematics, a complex number is an element of a number system that extends the real numbers with specific element denoted 'i', called the imaginary unit and satisfying the equation $i^2 = -1$ (or equivalently $i = \sqrt{-1}$). Every complex number z can be expressed in the form $z = a + bi$, where $a \in \mathbb{R}$ and $b \in \mathbb{R}$ are real numbers. Two complex numbers $z_1 = a_1 + b_1i$ and $z_2 = a_2 + b_2i$ are added according to the rule

$$z_1 + z_2 = (a_1 + a_2) + (b_1 + b_2)i$$

and multiplied by each other according to the rule

$$z_1 \cdot z_2 = (a_1 \cdot a_2 - b_1 \cdot b_2) + (a_1 \cdot b_2 + a_2 \cdot b_1)i.$$

Your task is to write C++ code to define a class `ComplexNumber`, its data members, member functions, and non-member functions (if there are) to reflect the above description.

1. Write code to define the class `ComplexNumber` with two private data members, named `re` and `im`.
2. Write a constructor receiving two input arguments `re` and `im` with default value 0 to initialize the data members described in (a). Use list initialization (not the assignment statements) to initialize these data members.

- Write a function overloading the operator `<<` of the object `std::ostream` so that the statement “`std::cout << comp_num;`” works for the `ComplexNumber` object `comp_num`. You must decide the print-out format that makes sense to the user. For example, consider the following numbers: $1 + 0i = 1$ and $0 + 2i = 2i$.
- Write functions to overload the operator `+` (plus sign) and the operator `*` (multiplication sign) to reflect the addition and multiplication of two complex numbers, respectively. Essentially, the following statements should work after overloading the operators `<<`, `+` and `*` for the complex numbers:

```
std::cout << a + b;
std::cout << a * b;
```

- Write a member function with name `pow` to compute the power z^n of the complex number z with n being a non-negative integer. Recall that

$$z^0 = 1, \quad z^1 = z, \quad z^2 = z \cdot z, \quad z^3 = z \cdot z \cdot z, \quad z^n = \underbrace{z \cdot \dots \cdot z}_{n \text{ terms}}.$$

- Write a function to overload the assignment operator `=` for a `ComplexNumber` object so that the following statement works:

```
c = a + b; // a, b, c are all ComplexNumber objects
```

- Write a class `RealNumber` derived from the class `ComplexNumber` you have defined above. The class `RealNumber` must have a constructor receiving one `double` number and make sure that the imaginary part derived from the parent class is equal to zero.
- Write a `main` function to perform the following tasks:

- Create an array of five `ComplexNumber` elements using dynamic memory allocation with a pointer to `ComplexNumber`. Remember to clean up the memory when you do not need the array anymore.
- Initialize these five complex numbers using the `for`-loop to obtain five numbers (note the pattern of the real parts and imaginary parts of these numbers):

$$z_1 = 1 + 2i, \quad z_2 = 2 + 3i, \quad z_3 = 3 + 4i, \quad z_4 = 4 + 5i, \quad z_5 = 5 + 6i.$$

- Compute the sum $s = \sum_{j=1}^5 z_j$ of these five numbers using the `for`-loop and print the resulted sum s to the console.
- With $z = 1 + 2i$, compute the expression

$$\Lambda = 1 + z + z^2 + z^3 + z^4 + z^5$$

using the `for`-loop.

Note that you can write any helper functions that are not defined by answering the above questions to complete the `main` function and other functions. [9 marks]

Hint: `static_cast<double>(42)` will convert the integer 42 into the double 42.0.

Answer in Code

```

1  #include <iostream>
2  #include <cmath>
3
4  class ComplexNumber {
5  private:
6      double re;
7      double im;
8
9  public:
10     // Constructor with list initialization and default values
11     ComplexNumber(double r = 0, double i = 0) : re(r), im(i) {}
12
13     // Overloading the assignment operator
14     ComplexNumber& operator=(const ComplexNumber& other) {
15         if (this != &other) {
16             re = other.re;
17             im = other.im;
18         }
19         return *this;
20     }
21
22     // Overloading the + operator
23     ComplexNumber operator+(const ComplexNumber& other) const {
24         return ComplexNumber(re + other.re, im + other.im);
25     }
26
27     // Overloading the * operator
28     ComplexNumber operator*(const ComplexNumber& other) const {
29         double new_re = (re * other.re) - (im * other.im);
30         double new_im = (re * other.im) + (im * other.re);
31         return ComplexNumber(new_re, new_im);
32     }
33
34     // Power function  $z^n$ 
35     ComplexNumber pow(int n) const {
36         ComplexNumber result(1, 0); //  $z^0 = 1$ 
37         for (int i = 0; i < n; ++i) {
38             result = result * (*this);
39         }
40         return result;
41     }
42
43     // Friend function to overload << for ostream
44     friend std::ostream& operator<<(std::ostream& os, const ComplexNumber& cn) {
45         if (cn.re != 0 && cn.im > 0) {
46             os << cn.re << " + " << cn.im << "i";
47         } else if (cn.re != 0 && cn.im < 0) {
48             os << cn.re << " - " << std::abs(cn.im) << "i";
49         } else if (cn.re == 0 && cn.im != 0) {
50             os << cn.im << "i";
51         } else {
52             os << cn.re;
53         }
54         return os;

```

```

55     }
56 };
57
58 // Derived class RealNumber
59 class RealNumber : public ComplexNumber {
60 public:
61     RealNumber(double r) : ComplexNumber(r, 0) {}
62 };
63
64 int main() {
65     // Dynamic allocation of an array of 5 ComplexNumber objects
66     ComplexNumber* z_array = new ComplexNumber[5];
67
68     // Initialize using for-loop
69     // Pattern: z_j = j + (j+1)i
70     for (int j = 0; j < 5; ++j) {
71         double r = static_cast<double>(j + 1);
72         double i = static_cast<double>(j + 2);
73         z_array[j] = ComplexNumber(r, i);
74     }
75
76     // Compute sum s = Σ z_j
77     ComplexNumber s(0, 0);
78     for (int j = 0; j < 5; ++j) {
79         s = s + z_array[j];
80     }
81     std::cout << "Sum s = " << s << std::endl;
82
83     // Compute Λ = 1 + z + z^2 + z^3 + z^4 + z^5 for z = 1 + 2i
84     ComplexNumber z(1, 2);
85     ComplexNumber lambda(0, 0);
86     for (int n = 0; n <= 5; ++n) {
87         lambda = lambda + z.pow(n);
88     }
89     std::cout << "Lambda = " << lambda << std::endl;
90
91     // Clean up dynamic memory
92     delete[] z_array;
93
94     return 0;
95 }

```

2.3 Question 3: UML Class Diagram

Problem description

Assume that we want to plot basic shapes such as **Polygon**, **Rectangle**, **Square**, **Ellipse**, **Circle**.

Your task is to design and draw a UML class diagram for 7 classes described below (**bold-faced words**) so that they incorporate the following features:

- A **Point** in 2-dimensional space has coordinates x and y .
- A **Polygon** receives a list of points of data type **Point** just defined above. A **Polygon** must have at least 3 points to constitute a proper **Polygon**.

- A **Rectangle** is a **Polygon**. A **Rectangle** receives a **Point** for its bottom-left corner, two real values for its width w and height h and one real value θ for its inclination angle measuring the positive smallest angle that one edge makes with the positive x -axis, measured counterclockwise.
- A **Square** is a **Rectangle** with the width being equal to the height.
- An **Ellipse** receives a **Point** for its center, two real values for its width w and height h and an angle inclination θ measuring the positive smallest angle that one axis makes with the positive x -axis, measured counterclockwise.
- A **Circle** is an **Ellipse** with the same width and height, which is the radius $r = w = h$.
- Clearly, all the above-mentioned basic shapes are **Shape**.
- External users cannot access data of a **Polygon** object, a **Rectangle** object, a **Square** object, an **Ellipse** object and a **Circle** object directly. External users can access coordinates directly. External users can use all the member functions of these objects.
- As we do not know any specific information about a general shape, we cannot draw it or store any data on it. However, it is conceptually important to tell the program that any specific **Shape** should be able to draw itself. For this reason, there must be a constraint that any (basic) shapes must have a member function (or method) **draw()** for them.

Tasks to perform:

1. Draw 7 classes with their relevant data members and methods.
2. Decide appropriate access specifiers (or access modifiers) of the variables and methods.
3. Decide the data types for data members, data types for input arguments and returns of member functions in each class and present them in the class diagram.
4. Decide the association relationships such as composition, aggregation, association between these classes. Express these relationships by the appropriate connections.
5. Decide the relationship cardinalities between Point and the classes that have association with Point, and show them on the diagram.
6. How do you enforce the “is a” relationship as described in the above problem description? Answer in words and show that in the diagram.
7. How do you enforce the condition that every basic shape, as described above, must have the method **draw()**? Answer in words and show that in the diagram.

Answer

This answer is generated by using **mermaid** with **quarto**.

